无锁设计4条原则

Posted on 2024-01-13 10:51:20 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2asjf2dLwxeLURwUt38Di1LrcYN)

简介

前面的文章介绍了无锁并发的相关知识，涵盖了无锁队列，无锁栈，环状无锁队列的设计，本文总结下无锁并发设计的几个设计规则，以供读者自己编写无锁代码时可以起到抛砖引玉的效果。

模型设计

原则1:

在原型设计中使用std::memory_order_seq_cst次序若代码服从std::memory_order_seq_cst次序，则对其进行分析和推理要比其他内存次序容易得多，因为它令全部操作形成一个确定的总序列。

回顾之前我们实现的无锁栈和无锁队列，它们的原始版本全都采用std::memory_order_seq_cst次序，当基本操作均正常工作后，我们才放宽内存次序约束。

在这种意义上，采用其他内存次序其实是一项优化，需要避免过早实施。我们通常只有先完全了解代码全貌，认清哪些代码操作核心数据结构，才可以确定放宽哪些操作的内存次序约束。否则，事情就会很棘手。即便代码在测试过程中正常工作，也无法保证在生产环境下代码依然如此，这令内存次序的放宽调整变得复杂。所以，仅仅测试代码的运行并不足够，除非我们能够采用测试工具（假如真的存在），系统化地核查线程访问内存次序的全部可能的组合，验证它们是否与指定的内存次序约束保持一致。

内存回收方案

原则2:

使用无锁的内存回收方案无锁代码中的一大难题是内存管理。最基本的要求是，只要目标对象仍然有可能正被其他线程指涉，就不得删除。然而，为了避免过度消耗内存，我们还是想

及时删除无用的对象。

我们在这一章学习了3种方法，以确保内存回收满足安全要求：

1 暂缓全部删除对象的动作，等到没有线程访问数据结构的时候，才删除待销毁的对象；

2 采用风险指针，以辨识特定对象是否正在被某线程访问；

3 就对象进行引用计数，只要外部环境仍正在指涉目标对象，它就不会被删除。

3种方法的关键思想都是以某种方式掌握正在访问目标对象的线程数目，仅当该对象完全不被指涉的时候，才会被删除。针对无锁数据结构，还有很多别的方法可以回收内存。

譬如，无锁数据是使用垃圾回收器的理想场景。若我们得以采用垃圾回收器，即事先知晓它具备适时删除无用节点的能力，则算法的实现代码写起来就会轻松一些。

另一种处理方法是重复使用节点，等到数据结构销毁时才完全释放它们。我们之前实现的环状无锁队列就是重复使用固定个数的队列，头尾成环。由于重用了节点，因此所分配的内存便一直有效，代码从而避开了一些涉及未定义行为的麻烦细节。然而，这种方法有一个缺点，它导致程序频频出现被称为“ABA问题”的情形。

防范ABA问题

原则3：防范ABA问题在所有涉及比较-交换的算法中，我们都要注意防范ABA问题。

该问题产生过程如下：

步骤1：线程甲读取原子变量x，得知其值为A。

步骤2：线程甲根据A执行某项操作，比如查找，或如果x是指针，则依据它提取出相关值（称为ov）。

步骤3：线程甲因操作系统调度而发生阻塞。

步骤4：另一线程对原子变量x执行别的操作，将其值改成B。

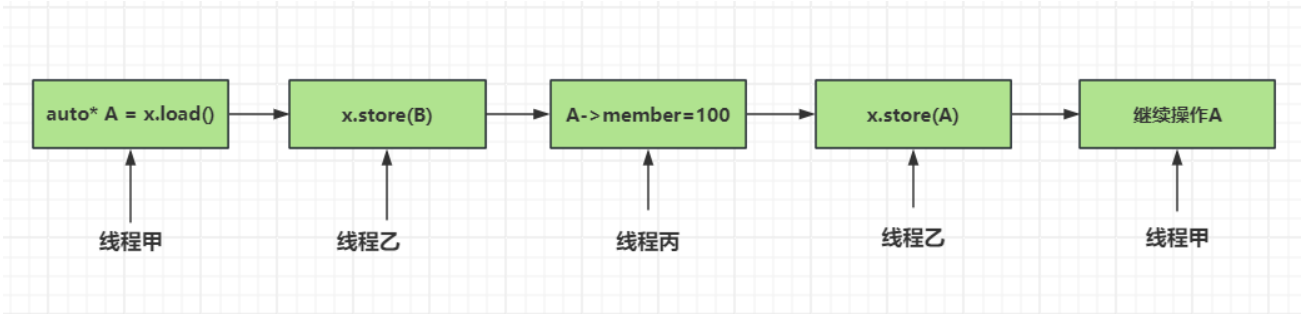
步骤5：又有线程改变了与A相关的数据，使得线程甲原本持有的值失效（步骤2中的ov）。这种情形也许是A表示某内存地址，而改动操作则是释放指针的目标内存，或变更目标数据，最后将产生严重后果。

步骤6：原子变量x再次被某线程改动，重新变回A。

若x属于指针型别，其指向目标可能在步骤5被改换成一个新对象。

步骤7：线程甲继续运行，在原子变量x上执行比较-交换操作，与A进行对比。因此比较-交换操作成功执行（因x的值依然为A），但A的关联数据却不再有效，即原本在步骤2中取得的ov已失效，而线程甲却无从分辨，这将破坏数据结构。

画出示意图



之前我们实现的无锁结构均不存在ABA问题，但它很容易由无锁算法的代码引发。该问题最常见的解决方法之一是，在原子变量x中引入一个ABA计数器。将变量x和计数器组成单一结构，作为一个整体执行比较-交换操作。每当它的值被改换，计数器就自增。照此处理，如果别的线程改动了变量x，即便其值看起来与最初一样，比较-交换操作仍会失败。

如果某数据结构的操作算法涉及空闲内存列表，或者涉及循环使用节点(比如我们之前实现的循环队列)，而不是通过内存分配器回收管理，那么ABA问题就格外常见。

举一个例子 原有的栈结构为 A —> B —> C

假设一个线程1执行pop将A头部节点的数据加载出来，还未做读改写更新head为B，此时时间片被其他线程2抢占执行pop将A,B分别出栈，然后线程3抢占时间片又将A入栈，那么我们看到此时栈的情况为 A —> C , 如果时间片切换回线程1，此时线程1执行读改写操作发现head还是为A，他会误认为这期间没有其他线程改变栈，所以线程1的读改写将head更新为B。其实B已经被弹出了，那么这就是ABA问题。

上面的仅仅是一个ABA案例的描述，那我们实现的无锁栈或者无锁队列为什么不存在这个ABA问题呢？原因是我们每次push加入的node节点都不同。node的结构包含数据域和下一个节点的指针

```
1. struct node
2. {
3.     std::shared_ptr<T> data;
4.     node* next;
5.     node(T const& data_) : //<--- - 1
6.         data(std::make_shared<T>(data_))
7.     {}
8. };
```

我们每次调用push虽然传递的data可以理解为A，但是构造的智能指针地址不同，也就是node中存储的data不同。所以即使线程1做读改写比较的时候发现数值都为A，但是地址不同，也可区分出栈被改变。

```
1. void push(T const& data)
2. {
3.     node* const new_node = new node(data);    //<--- - 2
4.     new_node->next = head.load();    //<--- - 3
5.     while (!head.compare_exchange_weak(new_node->next, new_node));    //<--- - 4
6. }
7.
8. std::shared_ptr<T> pop() {
9.     node* old_head = nullptr; //1
10.    do {
11.        old_head = head.load(); //2
12.        if (old_head == nullptr) {
13.            return nullptr;
14.        }
15.    } while (!head.compare_exchange_weak(old_head, old_head->next)); //3
16.
17.    return old_head->data;  //4
18. }
```

解决忙等

原则4：找出忙等循环，协助其他线程。

如我们在无锁队列第一版的push代码中，若两个线程同时执行压入操作，其中一个就须等待另一个结束，才可以继续运行。这实质上是一个忙等循环，如果放任不管，受到阻塞的线程就唯有浪费CPU时间却无计可施。阻塞型操作与使用互斥和锁一样，三者均有可能以忙等循

环的方式实现。

假设按照调度安排，某线程先开始执行，却因另一线程的操作而暂停等待，那么只要我们修改操作的算法，就能让前者先完成全部步骤，从而避免忙等，操作也不会被阻塞。

之后我们让比较失败的线程辅助完成节点的创建和tail的更新。这要求将非原子变量的数据成员改为原子变量，并采用比较-交换操作设置其值。不过，更复杂的数据结构需要进行更多修改。

浏览(2438) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) ♡喜欢(546) (JavaScript:void(0);)

热门评论

H B I

请输入正文

评论

没有更多评论了,点击收起 ≪ (JavaScript:void(0);)

热门文章

slice介绍和使用

♡喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

♡喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

♡喜欢(587) 浏览(1595)

解密定时器的实现细节

♡喜欢(566) 浏览(1449)

Linux环境搭建和编码

♡喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。
京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)
京ICP备2021033081号 (https://beian.miit.gov.cn/)